PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS
Compiladores
2s/2026 – Prof. Lucas Reis

Projeto MicroC – Etapa 7

Alocação de registradores e execução RV64

2º semestre de 2026

Objetivo

O grupo implementará as análises e a alocação que permitem executar sua pseudoRV64 com um conjunto finito de registradores. A entrada continua sendo o `ProgramIR` da Etapa 6. As instruções e a semântica da linguagem não mudam.

Resultado da etapa

pseudoRV64 ⟶ CFG e vivacidade ⟶ LRs globais
interferência
alocação ⟶ assembly RV64 ⟶ executável

O grupo constrói o CFG, calcula a vivacidade, identifica os LRs globais, constrói o grafo e implementa *simplify/select*. As partes fornecidas realizam spills, frames, convenção de chamadas, emissão, montagem, ligação e execução.

1 Organização e entrega

1.1 Recupere o projeto anterior

Reúna o frontend e o gerador da Etapa 6 diretamente em `previous/`. Copie os módulos antigos primeiro. Depois copie `codegen.py`, `selector.py`, `catalogue.py` e `patterns.py` da Etapa 6. Preserve os carregadores e a IR fornecidos nesta etapa. Não copie Git, ambientes virtuais, caches ou a pasta `previous/` como subdiretório. O CFG semântico continua no frontend. O novo CFG é construído sobre instruções.

1.2 Material inicial e entrega

Implemente a construção do CFG em `rv_cfg.py` e as duas análises de vivacidade em `liveness.py`. Implemente os LRs em `live_ranges.py`. Construa o grafo em `interference.py` e faça a coloração em `regalloc.py`. As funções pendentes lançam `NotImplementedError`. Auxiliares são permitidos. Preserve as interfaces públicas e não modifique as instruções, seus operandos ou os dados recebidos pelas análises.

Prazo desta etapa

Domingo, **29 de novembro de 2026, às 23h59**, no horário de Brasília. O commit deve estar no repositório correto do grupo no GitHub até esse horário.

2 Interfaces e representação

O backend recebe um `ProgramIR` validado da Etapa 6. Para cada `FunctionIR`, as passagens abaixo produzem resultados que alimentam a passagem seguinte. O coordenador e o emissor estão fornecidos.

```
cfg = build_cfg(function_ir)
live = solve_liveness(cfg)
points = solve_instruction_liveness(cfg, live)
ranges = build_live_ranges(cfg, points)
graph = build_interference(cfg, points, ranges)
allocation = allocate(graph)
```

Cada cadeia de chamadas trata uma função. VRegs são identificados pelo objeto completo, incluindo sua função proprietária. Não use somente o número impresso.

Estrutura	Campos e significado
`BasicBlock`	`start` e `stop`: índices inteiros. `predecessors` e `successors`: conjuntos de identificadores de blocos.
`CFG`	`function`: a `FunctionIR` recebida. `blocks`: dicionário de identificador para `BasicBlock`. `entry`: identificador da entrada ou `None`.
`Liveness`	`live_in` e `live_out`: dicionários de identificador de bloco para conjunto de VRegs.
Grafo	Dicionário LR $\mapsto$ conjunto de LRs vizinhos.
Alocação	Dicionário LR $\mapsto$ registrador ou `None`.

O intervalo do bloco é `instructions[start:stop]`. Seu identificador é `start`. Os mapas de vivacidade usam esses identificadores. O grafo inclui vértices isolados. Uma função sem instruções tem CFG vazio e entrada `None`.

Por exemplo, um bloco com `start=1` e `stop=4` contém as instruções de índices 1, 2 e 3. Ele está em `cfg.blocks[1]`, e seu conjunto de entrada está em `live.live_in[1]`. O número 4 não pertence ao intervalo.

Função	Entrada e retorno
`build_cfg`	Recebe `FunctionIR`. Retorna `CFG`, com blocos e conexões preenchidos.
`solve_liveness`	Recebe `CFG`. Retorna `Liveness` no ponto fixo.
`solve_instruction_liveness`	Recebe o CFG e a vivacidade por bloco. Retorna `InstructionLiveness`.
`build_live_ranges`	Recebe o CFG e a vivacidade por instrução. Retorna um mapa de pontos para LRs.
`build_interference`	Recebe o CFG, a vivacidade por instrução e os LRs. Retorna o dicionário de adjacência.
`allocate`	Recebe o grafo e o pool. Retorna uma decisão para cada LR.

Os tipos estão em `backend_api.py`. Use as instâncias fornecidas. O grupo pode criar estruturas auxiliares, preservando esses formatos de retorno. No grafo, cada chave representa um LR e seu valor é um conjunto de LRs vizinhos. Mesmo um LR sem vizinhos deve aparecer como chave, associado ao conjunto vazio.

3 Construção do CFG

Comece um bloco na primeira instrução, em cada `LabelInstr` e após branches, `Jump` e `Return`. Preserve labels e trechos inalcançáveis. Labels consecutivos originam blocos distintos, com continuação entre eles.

Final do bloco	Sucessores
Branch	Destino e próximo bloco textual, quando existente.
`Jump`	Somente destino.
`Return`	Nenhum.
Outra instrução	Próximo bloco textual, quando existente.

`Call` e impressão continuam no bloco. Os predecessores devem refletir exatamente as mesmas arestas. Não adicione nós de entrada ou saída artificiais. O destino de um branch é um objeto `Label`. Localize o índice da `LabelInstr` correspondente para encontrar o identificador do bloco de destino. O ID do label e o índice da instrução são informações diferentes.

4 Vivacidade até ponto fixo

`uses(i)` e `defs(i)` estão fornecidos e descrevem uma instrução. Os resumos $USE(B)$ e $DEF(B)$ descrevem um bloco inteiro. $USE(B)$ contém os VRegs lidos antes de qualquer definição do próprio VReg nesse bloco. $DEF(B)$ reúne todos os VRegs definidos no bloco.

Considere este trecho, processado de cima para baixo:

Instrução	`uses(i)`	`defs(i)`	$USE(B)$ acumulado	$DEF(B)$ acumulado
`li v1, 2`	$\varnothing$	$\{v1\}$	$\varnothing$	$\{v1\}$
`add v0, v0, v1`	$\{v0, v1\}$	$\{v0\}$	$\{v0\}$	$\{v0, v1\}$

O uso de `v1` na soma não entra em $USE(B)$, pois seu valor é produzido no próprio bloco. O valor anterior de `v0` precisa chegar pela entrada. Considere as leituras antes da escrita, inclusive quando o destino também é fonte.

$LIVEIN(B)$ representa os valores necessários antes da primeira instrução do bloco. $LIVEOUT(B)$ representa os valores necessários após sua última instrução, por algum caminho que continua no CFG. Inicialize todos os conjuntos vazios e repita as equações até não haver mudança:

$$LIVEOUT(B) = \bigcup_{S \in succ(B)} LIVEIN(S)$$

$$LIVEIN(B) = USE(B) \cup (LIVEOUT(B) - DEF(B))$$

Na saída, reúna as necessidades de todos os sucessores. Na entrada, acrescente os usos do bloco e preserve as necessidades de saída que o bloco não define. Uma atualização pode alterar as necessidades dos predecessores. Por isso uma única passagem não basta em geral. O ponto fixo ocorre quando nenhum conjunto de entrada ou saída muda durante uma passagem completa. Não é exigida uma ordem de visita nem uma quantidade específica de iterações.

4.1 Entrada e parâmetros

Não estabeleça $LIVEIN(entry) = \varnothing$ como condição de fronteira. Esse conjunto começa vazio, como os demais, mas pode crescer durante a análise. Uma saída sem sucessores tem $LIVEOUT = \varnothing$.

Em uma função completa, cada argumento recebido é introduzido por uma instrução `Param`. Ela define seu VReg de destino, sem ler VRegs:

```
param v0, 0
return v0
```

Nesse bloco, $USE = \varnothing$, $DEF = \{v0\}$ e $LIVEIN = \varnothing$. O retorno usa o valor produzido por `Param`. Não há uma leitura virtual implícita antes dessa definição.

Um teste isolado de análise também pode fornecer somente um trecho:

```
add v2, v0, v1
return v2
```

Aqui, a análise encontra $LIVEIN(entry) = \{v0, v1\}$, pois o trecho necessita desses valores ao começar. Isso não autoriza ler uma variável sem inicialização em um programa MicroC. O trecho serve para testar a análise. A verificação semântica e a geração de programas executáveis continuam com suas regras anteriores.

4.2 Exemplo

Considere as instruções abaixo. Os números à esquerda são índices da lista.

```
0: li v0, 3
1: L0:
2: addi v0, v0, -1
3: bnez v0, L0
4: return v0
```

O CFG possui `entry=0` e três entradas em `blocks`. O bloco 1 pode ser representado como:

```
BasicBlock(start=1, stop=4,
           predecessors={0, 1}, successors={1, 4})
```

O predecessor 1 é o próprio bloco, por causa do desvio de retorno ao label. Os resumos são $USE(0) = \varnothing$, $DEF(0) = \{v0\}$, $USE(1) = DEF(1) = \{v0\}$, $USE(4) = \{v0\}$ e $DEF(4) = \varnothing$.

Para ilustrar uma iteração, visite os blocos na ordem 0, 1, 4, usando cada atualização imediatamente. Na primeira passagem, a visita ao bloco 1 produz $LIVEIN(1) = \{v0\}$, por causa do uso no decremento. Como o bloco 0 já foi visitado, seu $LIVEOUT$ ainda está vazio nessa passagem. Na próxima, ele recebe $LIVEOUT(0) = LIVEIN(1) = \{v0\}$. O resultado estável é:

Bloco	Intervalo	Sucessores	LIVEIN	LIVEOUT
0	$[0, 1)$	$\{1\}$	$\varnothing$	$\{v0\}$
1	$[1, 4)$	$\{1, 4\}$	$\{v0\}$	$\{v0\}$
4	$[4, 5)$	$\varnothing$	$\{v0\}$	$\varnothing$

O valor é necessário na entrada do laço porque o decremento o lê antes de reescrevê-lo. A definição inicial elimina essa necessidade na entrada da função. Por exemplo, os resultados da última linha estão em `live.live_in[4]` e `live.live_out[4]`.

4.3 Vivacidade por instrução

```
points = solve_instruction_liveness(cfg, live)
```

O retorno tem tipo `InstructionLiveness` e contém os mapas `before` e `after`. Ambos usam o índice original da instrução e contêm um conjunto de VRegs para cada índice, inclusive quando o conjunto é vazio.

Em cada bloco, percorra as instruções de trás para frente, começando pelo $LIVEOUT$ do bloco. O conjunto disponível ao visitar uma instrução é $after[i]$. Calcule o conjunto de entrada pela equação

$$before[i] = USE(i) \cup (after[i] - DEF(i)).$$

O resultado vira o conjunto de saída da instrução anterior dentro do bloco. No laço do exemplo, antes e depois do decremento aparece `v0`. O conjunto anterior se refere ao valor lido e o posterior, ao valor escrito.

5 Construção dos LRs globais

Um intervalo de vivacidade (*live range*, LR) agrupa os pontos em que valores de um mesmo VReg precisam compartilhar uma localização. Uma nova definição pode iniciar outro LR. Em uma junção, definições de ramos diferentes podem integrar o mesmo LR porque chegam a um uso comum.

5.1 Pontos e conexões

`build_live_ranges` mapeia cada ponto `(i, phase, v)` para um LR. O ponto identifica o índice da instrução, a fase (`"before"` ou `"after"`) e o VReg original.

- Use os pontos vivos antes e depois de cada instrução. Inclua um ponto `"after"` para cada definição morta.
- Conecte os pontos antes e depois da mesma instrução para cada VReg vivo nos dois lados que a instrução não redefine.
- Conecte a saída de uma instrução à entrada da próxima para os VRegs vivos nos dois pontos. Faça isso dentro dos blocos e nas arestas do CFG.

Cada componente conexo recebe um `VReg` com o símbolo da função e um ID inteiro não negativo, distinto dos demais LRs da função. As conexões são não dirigidas. A correção compara os grupos de pontos, com IDs livres.

5.2 Exemplo

```
0: li v0, 1
1: print_int v0
2: li v0, 2
3: return v0
```

Um LR agrupa `(0, "after", v0)` e `(1, "before", v0)`. Outro agrupa `(2, "after", v0)` e `(3, "before", v0)`. A impressão consome o primeiro valor e o retorno usa o segundo. Os dois LRs podem receber localizações diferentes.

No exemplo do laço, os pontos antes e depois do decremento não são conectados diretamente, pois há uma redefinição. A aresta de retorno conecta o valor produzido à entrada do laço. Por esse caminho, os pontos pertencem ao mesmo componente. Em uma junção, conecte todas as entradas previstas pelo CFG, incluindo cada predecessor que fornece o valor vivo.

6 Grafo de interferência

Crie um vértice por LR encontrado, inclusive para definições mortas e LRs sem vizinhos. A passagem recebe o CFG, a vivacidade por instrução e o mapa de pontos.

6.1 Clique na entrada

Uma **clique** é um conjunto de vértices em que cada vértice está conectado a todos os demais. Para formar uma clique com $\{v0, v1, v2\}$, acrescentamos as três arestas $v0$–$v1$, $v0$–$v2$ e $v1$–$v2$. Neste projeto, os LRs dos pontos `"before"` na primeira instrução da entrada devem formar uma clique. Portanto, conecte cada par distinto desse conjunto. Se ele tiver zero ou um VReg, nenhuma aresta será acrescentada nessa etapa.

Essa regra cobre valores necessários desde o início do trecho, mesmo quando não há uma instrução anterior que os defina. Aplique-a somente à entrada da função. Não acrescente cliques em todos os conjuntos de vivacidade.

6.2 Interferência a partir dos pontos

Para cada definição na instrução i, consulte seu LR no ponto `"after"`. Conecte-o aos LRs vivos depois de i, exceto a si mesmo. Use `points.after[i]` para saber quais VRegs estão vivos e consulte seus LRs nos pontos `"after"` correspondentes. Em `Move(d,s)`, exclua também o LR do ponto `"before"` de `s` das arestas produzidas por essa instrução. A exceção não remove arestas já existentes.

Considere um bloco com o código abaixo e $LIVEOUT = \varnothing$:

```
li v0, 7
li v1, 2
mv v2, v0
add v3, v0, v1
add v3, v3, v2
return v3
```

Instrução visitada	Vivos depois	Novas arestas	Vivos antes
`return v3`	$\varnothing$	nenhuma	$\{v3\}$
`add v3,v3,v2`	$\{v3\}$	nenhuma	$\{v2, v3\}$
`add v3,v0,v1`	$\{v2, v3\}$	$v2$–$v3$	$\{v0, v1, v2\}$
`mv v2,v0`	$\{v0, v1, v2\}$	$v1$–$v2$	$\{v0, v1\}$
`li v1,2`	$\{v0, v1\}$	$v0$–$v1$	$\{v0\}$
`li v0,7`	$\{v0\}$	nenhuma	$\varnothing$

A tabela segue a ordem do cálculo da vivacidade por instrução, de baixo para cima no código. A tabela usa os nomes originais para abreviar. Há cinco vértices, pois as duas definições de `v3` pertencem a LRs diferentes. A aresta $v2$–$v3$ envolve o LR da primeira definição de `v3`. O LR da segunda definição fica isolado.

6.3 Cópias e definições mortas

No exemplo, `mv v2,v0` deixa fonte e destino com o mesmo valor. Embora ambos estejam vivos após a cópia, ela não acrescenta $v0$–$v2$. Isso permite que recebam o mesmo registrador, se nenhuma outra instrução produzir essa interferência. A exceção não remove uma aresta criada em outro ponto.

Uma definição morta também deve participar da construção. Em um trecho com `li v9,0` seguido de `return v0`, a escrita em `v9` não pode destruir o valor de `v0`. A regra acrescenta $v9$–$v0$, mesmo que `v9` não seja usado depois. A etapa não exige remover a instrução morta.

A IR possui no máximo uma definição por instrução. As arestas são simétricas, sem autoarestas. A correção compara conjuntos, ignorando ordem. Para comparação, uma aresta pode ser normalizada como `frozenset({u,v})`.

7 Alocação por coloração

7.1 Pool e contrato de spill

O pool de registradores é `s0` a `s8`, com $K = 9$. O scaffold não usa `s0` como frame pointer. Retorne exatamente uma decisão por vértice: nome de registrador do pool ou `None`. Não retorne slots ou offsets. Um LR marcado para spill reside em memória entre instruções. O scaffold carrega suas fontes e armazena suas definições. Essa política vale para VRegs mutáveis e cópias. O grupo não insere loads/stores nem reconstrói o grafo após spills. Por exemplo, se `v0` e `v1` forem objetos VReg, um resultado possível é:

```
{v0: "s0", v1: None}
```

O valor do LR `v0` permanece em `s0` durante sua vivacidade. O scaffold reserva um slot para `v1`, carrega seu valor quando uma instrução o lê e o armazena quando uma instrução o define. O mapa não contém o número desse slot. A mesma decisão vale para todos os pontos do LR. O scaffold consulta `"before"` para cada leitura e `"after"` para cada destino. Ele usa essas localizações na emissão e preserva a pseudoRV64 original.

7.2 Simplify e select

Simplify determina uma ordem de remoção. Trabalhe sobre uma cópia do grafo e preserve as adjacências originais. O grau residual de um nó conta somente seus vizinhos que ainda não foram removidos.

Se existir um nó de grau menor que K, remova um deles e empilhe-o. Quando esse nó voltar, seus vizinhos ainda presentes nessa remoção poderão ocupar no máximo $K - 1$ cores. Portanto, haverá uma cor livre para ele. Ao remover um nó, atualize os graus dos vizinhos que continuam ativos.

Quando nenhum nó tiver grau menor que K, escolha um candidato potencial a spill, remova-o e empilhe-o. Uma heurística simples, como maior grau, é suficiente. Essa escolha permite continuar a simplificação, mas ainda não determina o destino final do VReg.

Select atribui cores na ordem inversa. Desempilhe um nó e considere suas adjacências originais com os vizinhos já reintroduzidos. As cores usadas por esses vizinhos ficam indisponíveis. Vizinhos marcados para spill não ocupam cor. Atribua qualquer cor livre. Marque spill somente se nenhuma estiver disponível. **Um candidato potencial também deve tentar receber cor.** Não se exigem custos de laços, coalescing, splitting ou coloração ótima.

7.3 Exemplo de candidato que recebe cor

Para um exemplo pequeno, considere $K = 2$, cores `s0` e `s1`, e o ciclo de arestas A–B, B–C, C–D e D–A. As letras representam LRs. Inicialmente todos têm grau 2.

Remoção	Grau residual	Candidato potencial?	Pilha, base → topo
A	2	sim	A
B	1	não	A, B
C	1	não	A, B, C
D	0	não	A, B, C, D

Ao remover A, os graus de B e D caem para 1. A partir daí há nós de baixo grau. Uma seleção válida é:

Reinserção	Cores ocupadas pelos vizinhos já coloridos	Cor escolhida
D	nenhuma	`s0`
C	`s0`	`s1`
B	`s1`	`s0`
A	`s0`	`s1`

Os dois vizinhos de A, B e D, receberam a mesma cor. Assim, A consegue a outra cor e nenhum LR sofre spill. Escolhê-lo como candidato na simplificação não significou que faltaria cor na seleção.

7.4 Coletor usado pelos testes

```
allocate(graph, registers=REGISTERS, *, trace=None)
```

Quando `trace` for uma lista fornecida pelo teste, acrescente um par com o LR e um booleano a cada remoção. O booleano indica candidato potencial. O coletor não altera a decisão e não é um relatório de entrega. No exemplo anterior, a lista seria `[(A, True), (B, False), (C, False), (D, False)]`. Registre a ordem de remoção, não a ordem de reinserção. Quando `trace=None`, devolva o mesmo tipo de mapa final, sem exigir um coletor. Não altere a heurística em função de sua presença. O corretor reproduz os graus e a seleção a partir dessa ordem. Não exige uma heurística, ordem ou cor específica. Testes pequenos podem fornecer outro pool não vazio. O executável da MicroC sempre usa os nove registradores.

8 Execução, testes e critérios de conclusão

8.1 Ambiente

O ambiente é Python 3.12, GNU RISC-V e QEMU em modo usuário Linux. O scaffold gera RV64IM/LP64 estático, preserva os callee-saved usados, alinha a pilha a 16 bytes e trata argumentos excedentes e offsets grandes. O container reúne Python, bibliotecas de teste e toolchain. A partir da raiz do starter, construa a imagem uma vez e execute os testes das partes fornecidas:

```
docker build -t microc-rv64 .
docker run --rm -v "$PWD":/work microc-rv64 \
  python -m pytest -q tests/test_execution.py
```

Esses testes usam decisões de alocação preparadas pelo docente. Devem passar mesmo antes de implementar os cinco módulos. Uma falha nesse grupo deve ser investigada como problema do ambiente ou das partes fornecidas.

Em seguida, execute a suíte pública completa:

```
docker run --rm -v "$PWD":/work microc-rv64 \
  python run_tests.py
```

Os casos das funções pendentes falham inicialmente. Use essas falhas para acompanhar a implementação. Os testes de análise recebem IR fornecida, portanto podem ser usados antes de recuperar o gerador do grupo.

8.2 Inspeção e execução de um programa

Depois de recuperar o projeto anterior, execute o exemplo integrado:

```
docker run --rm -v "$PWD":/work microc-rv64 \
  python runner.py examples/integrado.mc --run
```

Para comparar as etapas, substitua a opção final conforme a tabela. Os arquivos de saída são criados na pasta compartilhada com o contêiner.

Opção do runner	Resultado
`--dump-ir`	Mostra a pseudoRV64 gerada. É o modo padrão.
`--run-ir`	Executa no interpretador, sem alocação física ou QEMU.
`--emit-asm programa.S`	Executa as análises e a alocação. Escreve o assembly RV64.
`--build programa`	Também monta e liga, produzindo o executável.
`--run`	Constrói e executa sob QEMU. Escreve somente a saída do programa.

Os comandos também podem ser executados diretamente com Python se o ambiente estiver instalado no host. `--run-ir` ajuda a distinguir um problema já presente na IR de um problema introduzido no backend. Os dois modos de execução devem produzir a mesma saída para um programa de comportamento definido.

8.3 Saída textual e valor de retorno

O interpretador retorna um objeto com `stdout` e `main_return`. Os testes comparam tanto o texto impresso quanto o valor integral de retorno. O retorno não é impresso automaticamente pelo runner.

O status normal de um processo conserva apenas os oito bits inferiores do retorno. Por exemplo, 300 produz status 44. Comparar somente esse status não distingue um retorno 300 de um retorno 44. Por isso o startup de teste captura os 64 bits em um canal separado, sem acrescentar texto a stdout. Esse mecanismo está fornecido. O grupo não implementa nem interpreta esse canal.

8.4 Critérios públicos e ocultos

Os testes públicos apresentam exemplos das regras de cada passagem. Os ocultos verificam essas mesmas regras em outras entradas e combinações. A ordem de remoção e as cores podem variar dentro das regras do algoritmo.

Componente	Critério
CFG	Blocos, intervalos e conjuntos exatos de predecessores e sucessores.
Vivacidade	Conjuntos exatos por bloco e antes/depois de cada instrução.
LRs globais	Pontos exatos e mesma partição em componentes. Os IDs dos LRs são livres.
Interferência	Vértices e arestas exatos segundo a regra deste enunciado.
Simplify/select	Remoções compatíveis com os graus e seleção válida, verificadas com o coletor.
Alocação	Uma decisão por vértice, registradores permitidos e cores diferentes em vértices adjacentes alocados.
Execução	Saída textual e retorno integral iguais aos do interpretador.

A ordem dos elementos em conjuntos e dicionários não é critério de correção. Para a quantidade de spills, valem os seguintes casos:

- Grafos que podem ser completamente simplificados removendo nós de grau menor que K devem terminar sem spills. Com o pool de nove registradores, isso inclui florestas e cliques com até nove vértices. O ciclo com duas cores apresentado no exemplo também deve terminar sem spills.
- Com o pool de nove registradores, uma clique de nove vértices deve ter zero spills. Uma clique de dez exige dez cores para manter todos os vértices em registradores. Nesse caso, a alocação deve ter exatamente um spill, sem exigir qual LR será escolhido.
- Em outros grafos, não se exige uma coloração ótima nem ausência de spills em todo caso matematicamente colorível. A decisão deve respeitar a seleção: um nó só recebe spill quando nenhuma cor está disponível ao reinseri-lo.

Assim, marcar todos os LRs como spill não atende aos casos que exigem zero spills. Escolher automaticamente spill para todo candidato potencial também não atende ao exemplo do ciclo.

Os testes do backend usam IR fornecida para isolar falhas da Etapa 6. A execução de exemplos MicroC pelo runner verifica adicionalmente a integração com o frontend e o gerador recuperados pelo grupo. Testes das partes fornecidas usam alocações forçadas,

inclusive todos os VRegs em memória. Eles verificam a materialização física e não são um critério de qualidade da alocação dos alunos.

Antes da entrega, confira o ponto fixo, as arestas simétricas, a tentativa de coloração dos candidatos e os testes públicos. Não oculte falhas com skip ou xfail.

9 Referências para consulta

Consulte a especificação MicroC e o enunciado da Etapa 6 para a semântica da IR. As Aulas 9 e 16 apresentam CFG, fluxo de dados e alocação global. Cooper e Torczon, *Engineering a Compiler*, 3ª edição, capítulo 13, seção 13.4, apresenta a alocação por coloração. A Aula 17 situa montagem, ligação e execução. A ABI e o runtime são partes fornecidas nesta entrega.